

CampusEats

Services, Contracts & Schema — Design Benchmark

1 Task 1 — Capabilities

The CampusEats system has six main jobs, called capabilities. Each capability does one clear type of work. These capabilities are later grouped into services based on the data they use.

#	Capability	What it does
1	Manage profile & delivery addresses	Students can create an account, log in, and manage the addresses where their food should be delivered.
2	Browse restaurants & menus	Students can see the available restaurants, their menus, and the prices of the food.
3	Add items to cart & place orders	Students can add food to a cart and then place an order.
4	Process payments & refunds	Students can pay for an order online. If something goes wrong, the money can be returned.
5	Assign a rider & track delivery	A rider is assigned to the order, and the student can see the delivery progress.
6	Send order notifications	The student gets a notification when the order is placed, paid, on the way, and delivered.

These six capabilities are handled by six services:

Accounts, Catalogue, Orders, Payments, Delivery, and Notifications.

Each service has its own data and does not directly share its database with another service.

See `services.drawio` / `services.png` for the service boundary diagram and the calls between services.

Services & Ownership

This is a solo project — one developer (You) designs, implements, and owns all six services.

Even though there is one developer, each service still keeps its own data separate from the other services.

Service	Owns	Responsible member
Accounts	users, addresses, login credentials	Kartik Karnwal
Catalogue	restaurants, menu items, prices	Kartik Karnwal
Orders	carts, orders, order status	Ankur Gupta
Payments	transactions, refunds	Kirti Gautam
Delivery	riders, assignments	Kirti Gautam
Notifications	message log	Pintu

2 Task 3 — Service Contracts

A service contract tells other services how they can use a service.

For each operation, the contract shows:

- what information must be sent,
- what information is returned, and
- what errors can happen.

The contract does not show database tables or how the service works internally.

Orders and Catalogue are the two main services and each has at least three operations. Every other service has at least one operation.

Contract: Orders (main service)

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cart	item unavailable
placeOrder	studentId, items, deliveryAddressId, paymentMethodId	orderId, status, total, estimatedMinutes	empty cart, bad address, item unavailable, payment declined
getOrder	orderId	order + status	not found
cancelOrder	orderId	status: cancelled	too late to cancel

Contract: Catalogue (main service)

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	—
getMenu	restaurantId	menu items + prices	not found
checkItem	itemId	available?, price	not found

Contract: Accounts

Operation	Input	Output	Errors
login	email, password	sessionToken, userId	invalid credentials
manageProfile	userId, updates	updated profile	invalid field
getAddress	addressId	address	not found

Contract: Payments

Operation	Input	Output	Errors
charge	orderId, amount, paymentMethodId	receipt, transactionId	payment declined
refund	transactionId, amount, reason	refundId, status	transaction not found, already refunded

Contract: Delivery

Operation	Input	Output	Errors
assignRider	orderId, addressId	assignmentId, riderName, eta	no rider available
trackDelivery	orderId	riderLocation, status	not found

Contract: Notifications

Operation	Input	Output	Errors
send	userId, type, message	messageId, status: sent	invalid user

3 Task 4 — Full Specification: `placeOrder`

`placeOrder` is the main operation in CampusEats. It takes the student's food choices and turns them into a real order.

Orders controls this process and asks the other services for the information it needs.

<code>placeOrder</code>	
IN	• <code>studentId</code> • <code>items: [{itemId, qty}]</code> • <code>deliveryAddressId</code> • <code>paymentMethodId</code>
OUT	• <code>orderId</code> • <code>status: 'placed'</code> • <code>total</code> • <code>estimatedMinutes</code>
ERRORS	• <code>item unavailable</code> • <code>payment declined</code> • <code>invalid address</code> • <code>empty cart</code>
HIDDEN FROM CALLERS	• the <code>orders/order_items</code> tables and how they are stored • how <code>orderId</code> is created • which rider is selected and how the rider is selected • which payment gateway is used inside <code>charge()</code> • how the total price is calculated

Flow:

The student sends a `placeOrder` request to Orders.

Orders then:

1. asks Catalogue using `checkItem` to check whether the item is available and get its price;
2. asks Payments using `charge` to take the payment;
3. asks Delivery using `assignRider` to arrange the delivery;
4. asks Notifications using `send(userId, type, message)` to send a notification;
5. returns the order confirmation to the student.

Orders controls the whole process and owns its carts, orders, and order status. It gets information that belongs to other services by using their contracts instead of directly accessing their data.

4 Task 6 — Validating the Design

Each service is checked using five properties:

1. reachable over a network,
2. keeps its own data,
3. has a clear contract,
4. can work independently, and
5. has loose connections with other services.

Service	Reach.	Self-cont.	Has contract	Independent	Loosely coupled
Accounts	✓	✓	✓	✓	✓
Catalogue	✓	✓	✓	✓	✓
Orders	✓	✓	✓	✓	✓
Payments	✓	✓	✓	✓	✓
Delivery	✓	✓	✓	(!) See note below.	✓
Notifications	✓	✓	(!) See note below.	✓	✓

Notes on the properties marked (!)

- **Delivery — Independent:**

Delivery may use a third-party maps or routing service to help choose a rider. If that outside service stops working, `assignRider` could also stop working.

Fix: Keep the outside service behind Delivery’s own contract. Add a timeout and a backup option, such as choosing the first available rider manually. This allows Delivery to keep working even if the outside service has a problem.

- **Notifications — Has a contract:**

Notifications has one operation, `send`. This is enough for the current design, but it is a small contract.

If the system later needs to check whether a notification was delivered, an operation such as `getStatus(messageId)` can be added.

Overall: All six services can be reached, keep their own data, and are loosely connected. Each service also has a contract. Delivery and Notifications have the two small points explained above, but both can be improved later without changing the main design.